DAYANANDA SAGAR COLLEGE OF ENGINEERING
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560078
(An Autonomous Institution affiliated to VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Module 3

Polymorphism

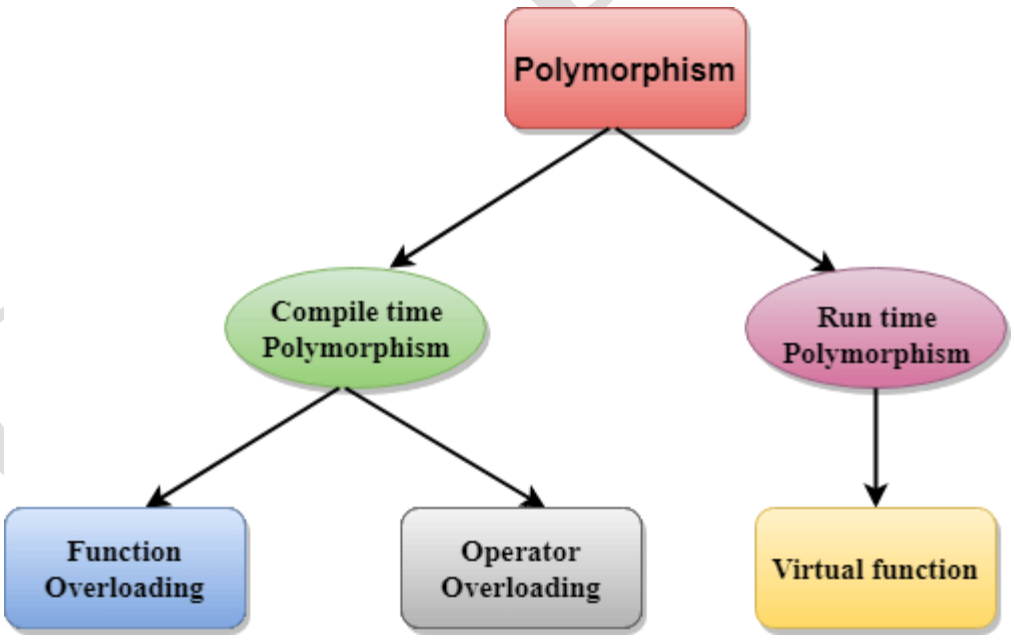
The term "Polymorphism" is the combination of "poly" + "morphs" which means having many forms. It is a greek word. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

Real Life Example of Polymorphism

Let's consider a real-life example of polymorphism. A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market. Here, a single person is behaving differently according to the situations.

There are two types of polymorphism in C++: Compile time Polymorphism

Runtime Polymorphism



1. Compile time polymorphism:

- The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time.
- It is achieved by **function overloading** and **operator overloading** which is also known as static binding or early binding.

Function Overloading: When there are multiple functions with same name but different parameters then these functions are said to be overloaded. Functions can be overloaded by change in number of arguments or/and change in type of arguments.

// C++ program for function overloading

```
#include <iostream >
using namespace std;
class Geeks
{
public:
// function with 1 int parameter
void func(int x)
{
cout << "value of x is " << x << endl;
}
// function with same name but 1 double parameter
void func(double x)
{
cout << "value of x is " << x << endl;
}
// function with same name and 2 int parameters
void func(int x, int y)
{
cout << "value of x and y is " << x << ", " << y << endl;
}
};
int main() {
Geeks obj1;
// Which function is called will depend on the parameters passed
```

```
// The first 'func' is called
obj1.func(7);
// The second 'func' is called
obj1.func(9.132);
// The third 'func' is called
obj1.func(85,64);
return 0;
}
```

Output:

```
value of x is 7
value of x is 9.132
value of x and y is 85, 64
```

In the above example, a single function named func acts differently in three different situations which is the property of polymorphism.

Operator Overloading: C++ also provide option to overload operators.

For example1, we can make the operator (‘+’) for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator ‘+’ when placed between integer operands , adds them and when placed between string operands, concatenates them.

```
#include<iostream>
using namespace std;
class Complex {
private:
int real, imag;
public:
Complex(int r = 0, int i =0) {real = r; imag = i;}
// This is automatically called when '+' is used with
// between two Complex objects
Complex operator + (Complex const &obj) {
Complex res;
res.real = real + obj.real;
res.imag = imag + obj.imag;
return res;
}
void print() { cout << real << " + i" << imag << endl; }
};
int main()
```

```
{
Complex c1(10, 5), c2(2, 4);
Complex c3 = c1 + c2; // An example call to "operator+"
c3.print();
}
```

Output:

12 + i9

In the above example the operator '+' is overloaded. The operator '+' is an addition operator and can add two numbers(integers or floating point) but here the operator is made to perform addition of two imaginary or complex numbers.

// program2 of function overloading when number of arguments vary.

```
#include <iostream>
using namespace std;
class Cal {
   public:
static int add(int a,int b){
      return a + b;
   }
static int add(int a, int b, int c)
   {
      return a + b + c;
   }
};
int main(void) {
   Cal C;                                  //  class object declaration.
   cout<<C.add(10, 20)<<endl;
   cout<<C.add(12, 20, 23);
  return 0;
}
```

Output:

30

55

Let's see the simple example when the type of the arguments vary.

CSE, DSCE

// **Program3** of function overloading with different types of arguments.

```
#include<iostream>
using namespace std;
int mul(int,int);
float mul(float,int);
int mul(int a,int b)
{
   return a*b;
}
float mul(double x, int y)
{
   return x*y;
}
int main()
{
   int r1 = mul(6,7);
   float r2 = mul(0.2,3);
   std::cout << "r1 is : " <<r1<< std::endl;
   std::cout <<"r2 is : "  <<r2<< std::endl;
   return 0;
}
```

Output:

r1 is : 42

r2 is : 0.6

Operations that can be performed:

Arithmetic operations: + – * / %

Logical operations: && and ||

Relational operations: == != >= <=

Pointer operators: & and *

Memory management operator: new, delete []

CSE, DSCE

Which operators Cannot be overloaded?

- Conditional [?:], size of, scope(::), Member selector(.), member pointer selector(.*) and the casting operators.
- We can only overload the operators that exist and cannot create new operators or rename existing operators.
- At least one of the operands in overloaded operators must be user-defined, which means we cannot overload the minus operator to work with one integer and one double. However, you could overload the minus operator to work with an integer and a mystring.
- It is not possible to change the number of operands of an operator supports.
- All operators keep their default precedence and associations (what they use for), which cannot be changed.
- Only built-in operators can be overloaded.

Advantages of an operator overloading in C++

- Simplified Syntax: Operator overloading allows programmers to use notation closer to the target domain, making code more intuitive and expressive.
- Consistency: It provides similar support to built-in types for user-defined types, enhancing consistency and ease of use.
- Easier Understanding: Operator overloading can make programs more accessible to understand by allowing the use of familiar operators with user-defined types, which can improve code readability and maintainability.

Difference between function overloading and overriding in C++?

	Function Overloading	**Function Overriding**
Definition	When two or more methods in a class have distinct parameters but the same method name, this is known as function overloading.	One method is in the parent class and the other is in the child class when a function is overridden, but they have the same parameters and method name.
Function signature	There should be a difference in the number or kind of parameters.	The function signature should not change.
Behaviour	defines several methods' behaviours.	changes the way the procedure behaves.

Scope of Function	They belong to the same category.	They have a distinct range.
Inheritance	Even without inheritance, it can happen.	Only when a class inherits from another does it happen
Polymorphism	Compile Time	Run Time
Technique	code improvement method.	method for replacing code.

Run time polymorphism: Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by **method overriding** which is also known as dynamic binding or late binding.

```
// C++ program for function overriding
#include <iostream>
using namespace std;
class base
{
public:
virtual void print ()
{ cout<< "print base class" <<endl; }
void show ()
{ cout<< "show base class" <<endl; }
};
class derived:public base
{
public:
void print () //print () is already virtual function in derived class,
 //we could also declared as virtual void print () explicitly
```

```
{ cout<< "print derived class" <<endl; }
void show ()
{ cout<< "show derived class" <<endl; }
};
//main function
int main()
{
base *bptr;
derived d;
bptr = &d;
//virtual function, binded at runtime (Runtime polymorphism)
bptr->print();
// Non-virtual function, binded at compile time
bptr->show();
return 0;
}
```

Output:

```
print derived class
show base class
```

Explanation: Runtime polymorphism is achieved only through a pointer (or reference) of base class type. Also, a base class pointer can point to the objects of base class as well as to the objects of derived class. In above code, base class pointer 'bptr' contains the address of object 'd' of derived class.

Late binding(Runtime) is done in accordance with the content of pointer (i.e. location pointed to by pointer) and Early binding(Compile time) is done according to the type of pointer, since

print() function is declared with virtual keyword so it will be bound at run-time (output is print derived class as pointer is pointing to object of derived class) and show() is non virtual so it will be bound during compile time(output is show base class as pointer is of base type).

NOTE: If we have created a virtual function in the base class and it is being overridden in the derived class then we don't need virtual keyword in the derived class, functions are automatically considered as virtual functions in the derived class.

C++ virtual function

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Late binding or Dynamic linkage

In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.

- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

```
#include <iostream>
using namespace std;
class A
{
int x=5;
public:
void display()
{
std::cout << "Value of x is : " << x<<std::endl;
}
};
class B: public A
{
int y = 10;
public:
void display()
{
std::cout << "Value of y is : " <<y<< std::endl;
}
};
int main()
{
A *a;
B b;
a = &b;
a->display();
return 0;
```

CSE, DSCE

```
}
```

Output:

Value of x is : 5

In the above example, * a is the base class pointer. The pointer can only access the base class members but not the members of the derived class. Although C++ permits the base pointer to point to any object derived from the base class, it cannot directly access the members of the derived class. Therefore, there is a need for virtual function which allows the base pointer to access the members of the derived class.

C++ virtual function Example

Let's see the simple example of C++ virtual function used to invoked the derived class in a program.

```
#include <iostream>
{
 public:
 virtual void display()
 {
  cout << "Base class is invoked"<<endl;
 }
};
class B:public A
{
 public:
 void display()
 {
  cout << "Derived Class is invoked"<<endl;
 }
};
int main()
{
 A* a;    //pointer of base class
 B b;     //object of derived class
```

CSE, DSCE

```
 a = &b;
 a->display();   //Late Binding occurs
}
```

Output:

Derived Class is invoked

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure virtual function can be defined as:

```
virtual void display() = 0;
```

Let's see a simple example:

```
#include <iostream>
using namespace std;
class Base
{
   public:
   virtual void show() = 0;
};
class Derived : public Base
{
   public:
```

```
    void show()
    {
        std::cout << "Derived class is derived from the base class." << std::endl;
    }
};
int main()
{
    Base *bptr;
    //Base b;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

Output:

Derived class is derived from the base class.

In the above example, the base class contains the pure virtual function. Therefore, the base class is an abstract base class. We cannot create the object of the base class.

Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.

It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

CSE, DSCE